Eleição de Líder, Replicação Primário-Backup e Detecção de Falhas em um Armazenamento Chave-Valor Distribuído

Caique Pinheiro Andrade (RA 204677) e Luiz Henrique Marques Gonçalves (RA 183218)
Instituto de Computação – Universidade Estadual de Campinas
MC714 – Sistemas Distribuídos – 2º semestre de 2026
c204677@dac.unicamp.br, l183218@dac.unicamp.br
Repositório (GitLab da disciplina): **https://gitlab.ic.unicamp.br/TODO**
Vídeo de explicação e demonstração: **https://TODO-video**

***Resumo*—Este trabalho implementa, do zero e sobre TCP com `asyncio`, um armazenamento chave-valor replicado em N nós que integra três algoritmos: eleição de líder pelo Valentão (*Bully*), replicação síncrona primário-backup e detector de falhas por *heartbeat*. Declaramos, para cada algoritmo, o modelo temporal, os canais, as falhas toleradas e as propriedades de segurança e vivacidade. Um commit exige quórum de maioria, épocas persistidas e uma fase de sincronização pós-eleição; juntos, eles impedem a perda de escritas confirmadas, inclusive sob partição. As propriedades são verificadas por 50 testes automatizados e por um verificador de invariantes sobre logs estruturados. Em uma campanha de 10 réplicas por configuração, sobre processos locais, o Valentão custou exatamente N mensagens no melhor caso e $N(N-1)$ no pior (igual à teoria, desvio zero), enquanto o tempo de eleição ($\approx 0{,}3$ s) não cresceu com N; uma escrita custou exatamente $2(N-1)$ mensagens; a indisponibilidade após o crash do primário foi de $1{,}3$ a $3{,}1$ s conforme o *timeout* do detector; e um primário isolado confirmou 0 escritas. Em 310 réplicas de falha, houve 0 violações de invariantes e 0 escritas confirmadas perdidas.**

I. Introdução e motivação

Serviços replicados precisam responder a duas perguntas: quem coordena e como as réplicas concordam sobre o que foi escrito. Um serviço de armazenamento tolerante a falhas combina, naturalmente, três mecanismos: um detector de falhas que suspeita de nós silenciosos, uma eleição que escolhe um novo coordenador e um protocolo de replicação que impede que uma escrita confirmada ao cliente seja perdida na troca. Escolhemos integrá-los em um só sistema, em vez de implementá-los isoladamente, porque as interações entre eles (falsas suspeitas provocando eleições, partições produzindo dois coordenadores) são onde estão os problemas de correção.

Os algoritmos são o Valentão [2] (eleição, grupo 1 do enunciado), a replicação primário-backup [3], [1] (replicação, grupo 2) e um detector por *heartbeat* [4] (terceiro algoritmo, bônus). A Seção II fixa o modelo do sistema; a Seção III descreve a arquitetura; a Seção IV apresenta os algoritmos e suas propriedades; a Seção V trata dos testes; as Seções VI e VII, do desenho experimental e dos resultados; e as Seções VIII e IX, das limitações e conclusões.

Código reutilizado. Não houve código reutilizado: toda a implementação, incluindo o transporte, foi escrita para este trabalho, em Python 3.11 e apenas com a biblioteca padrão nos nós. `pytest`, `numpy` e `matplotlib` são usados só em testes e experimentos. As referências são algorítmicas: o Valentão segue Garcia-Molina [2]; a replicação, Budhiraja et al. [3] e o capítulo 7 de van Steen e Tanenbaum [1]; o detector, a noção de detectores não confiáveis de Chandra e Toueg [4]. O par (época, posição) usado para ordenar réplicas é análogo ao termo e ao índice do Raft [5], que não foi usado como código.

II. Modelo do sistema

Nós e ambiente. N nós (padrão $N = 5$, ids $1, \ldots, N$) executam como processos independentes, cada um escutando em uma porta TCP e comunicando-se *somente* pela rede da aplicação. O mesmo código roda como processos locais ou como contêineres (`compose.yaml`); a diferença é só configuração (lista de pares, diretórios, temporizações). A máquina de desenvolvimento (Windows 11) não tem Docker, então todos os experimentos foram executados como processos locais; o `compose.yaml` e o `Dockerfile` são fornecidos para o ambiente de referência, mas *não foram executados* por nós.

Modelo temporal: parcialmente síncrono. Assumimos que, depois de um tempo desconhecido, as mensagens entre nós corretos são entregues em menos de τ_{rpc} e que o silêncio por τ_{fd} indica falha. Em um sistema assíncrono puro, eleição e detecção de falhas não têm solução determinística (FLP); o modelo síncrono é irreal, pois atrasos de escalonamento e de rede violam qualquer limite fixo. O modelo intermediário permite escolher temporizações e, quando violadas, o sistema degrada de forma controlada: perde *vivacidade* (falsas suspeitas, eleições desnecessárias), mas não *segurança*, que não depende dos tempos.

Canais. Cada mensagem usa uma conexão TCP própria, com uma linha JSON. Logo, a entrega é confiável e sem duplicação *enquanto o par está vivo*; *não há ordenação* entre mensagens distintas (conexões diferentes), e os protocolos toleram reordenação: backups aplicam por número de sequência

e detectam lacunas. Mensagens podem ser perdidas por queda do par ou pela camada de injeção de falhas (descarte, atraso fixo mais *jitter*, partição por conjunto de pares bloqueados), aplicada na aplicação, configurável em tempo de execução e semeada por nó. Toda chamada tem *timeout* explícito (`asyncio.wait_for`). A retransmissão é idempotente, e a duplicação eventual é absorvida pela idempotência de `REPLICAR` (Seção IV-C).

Falhas. Crash-stop e crash-recovery (um nó reiniciado recarrega época, líder reconhecido, *snapshot* e *log* de escritas do disco), perda e atraso de mensagens e partições de rede. Falhas bizantinas não são toleradas. O crash é produzido por `Popen.kill()`.

Relógios e persistência. Só relógios monotônicos locais, usados para *timeouts*; não há sincronização entre nós. Os *timestamps* dos logs servem apenas à análise, na mesma máquina. Cada nó mantém em disco `meta.json` (época e líder reconhecido), `snapshot.json` e um WAL; o `flush` a cada operação preserva os dados diante de `kill` e a opção `-fsync` cobre queda da máquina. Um nó reiniciado dispara eleição e só retorna à visão de replicação depois de re-sincronizar com o primário.

Contêineres não são domínios de falha independentes. Nós na mesma máquina compartilham kernel, disco, relógio e energia. Eles são nós *lógicos*: falhas correlacionadas (queda do hospedeiro), latências de rede reais e perdas de disco não são exercitadas, e as medidas de tempo refletem o escalonador de uma única máquina.

III. Arquitetura

O nó (`no.py`) compõe três módulos de protocolo que registram seus tratadores de mensagem: `bully.py`, `heartbeat.py` e `replicacao.py`. A máquina de estado da réplica (`replica.py`) é independente de rede, o que a torna testável em unidade. Os demais módulos são: `transporte.py` (TCP, *timeouts*, contadores de mensagens por tipo), `falhas.py` (injeção), `persistencia.py` (meta, snapshot, WAL), `logs.py` (JSONL), `verificador.py`, `cliente.py` e `cluster.py` (orquestração local por `Popen`). O envelope JSON de toda mensagem contém tipo, origem, destino, *época*, id, *incarnação* do remetente (distingue reinício de falsa suspeita) e o payload. Os contadores de enviadas e recebidas por tipo alimentam a métrica de complexidade de mensagens.

O estado compartilhado do nó é a *época* (maior época vista) e o *líder reconhecido nessa época*, persistidos a cada mudança. Toda mensagem de líder (`COORDENADOR`, `HEARTBEAT`, `REPLICAR`, `SINC_CONSULTA`) passa por uma regra única de aceitação: época menor $\Rightarrow$ recusa (`RECUSA`, que informa a época atual, é o *fencing*); mesma época $\Rightarrow$ só o líder já reconhecido (ou nenhum ainda); época maior $\Rightarrow$ aceita, e um líder que a recebe renuncia. Como o par (época, líder) é persistido, um nó que reinicia não aceita dois líderes diferentes na mesma época; é o papel do "voto" no Raft. O cliente fala com qualquer nó; um não primário responde `nao_lider` com o líder conhecido, e o cliente repete a operação durante *failovers* até um prazo.

IV. Algoritmos, hipóteses e correção

A. Eleição Valentão (Bully)

Hipóteses. Parcialmente síncrono: um `OK` chega em menos de τ_{rpc} se o nó está vivo. Canais como na Seção II; falhas crash-stop e crash-recovery com época persistida. Não há exigência de maioria, pois o Valentão original assume que, na ausência de partição, a ausência de resposta indica crash; sob partição essa hipótese falha, o que tratamos adiante.

Funcionamento. Quem (re)inicia ou suspeita do líder envia `ELEICAO` aos ids maiores e espera `OK` por τ_{rpc}. Sem `OK`, vence: persiste a época $\max(\text{épocas vistas}) + 1$ e anuncia `COORDENADOR`. Com `OK`, espera `COORDENADOR` (ou um *heartbeat* de líder válido) por $\tau_{coord} = 4\tau_{rpc}$ e, esgotado, recomeça. Se o líder atual recebe `ELEICAO`, responde `OK` e envia um *heartbeat* em vez de reeleger, o que evita trocar de época por causa de uma falsa suspeita de um seguidor. Quem vê um líder de id menor que o seu dispara eleição: o maior id vivo acaba líder.

Segurança e vivacidade. *Segurança*: no máximo um líder *efetivo* (que chega a servir escritas) por época; mensagens de época antiga são recusadas; a época de cada nó nunca diminui. *Vivacidade*: sob sincronia parcial e com a maioria dos nós vivos, todos os nós vivos concordam eventualmente no mesmo líder, o de maior id vivo.

Observação da verificação. Quando o nó de maior id volta de um crash, ele anuncia `COORDENADOR` com época $\max+1$, que pode colidir com a do líder interino. A regra de aceitação (um líder por época, persistido) recusa o anúncio; o recém-chegado recebe a `RECUSA`, renuncia e reelege-se em época maior. O resultado é seguro e custa uma rodada extra.

Partição e *split-brain*. O Valentão não exige maioria: com a rede particionada, cada lado pode eleger um "líder", e ambos podem usar a mesma época $\max+1$. Isso é tolerado porque *eleger não é poder servir*: o líder só serve escritas depois da fase de sincronização (Seção IV-C), que exige respostas de um quórum e *promete* a época (quem responde passa a recusar primários de época menor). Como só um lado tem maioria, só ele torna seu líder *efetivo*; o outro permanece "não pronto". O primário antigo, isolado, perde o quórum, não confirma e renuncia ao ver a época maior.

B. Detector de falhas por heartbeat

Hipóteses. Parcialmente síncrono (o detector só é *eventualmente preciso* se existirem limites de atraso; é não confiável por construção); perda e atraso por injeção; falhas crash e partições. O detector pertence à classe "eventualmente perfeito" apenas sob sincronia parcial [4]; sem ela, é só um suspeitador.

Funcionamento. O líder envia `HEARTBEAT` a cada τ_{hb} (uma tarefa por par, para que um par lento não atrase os demais), com época, líder, visão, `pronto` e posição; o par responde `HB_ACK`. Um seguidor sem heartbeat do líder por τ_{fd} suspeita e dispara eleição; o líder sem resposta de um membro da visão por τ_{fd} o remove da visão. Cada suspeita é registrada; se o

suspeito volta a falar com a mesma incarnação, a suspeita é marcada como *falsa* (com incarnação diferente, foi um reinício real).

Segurança e vivacidade. *Completude* (todo nó que sofreu crash passa a ser suspeito por todos os corretos, se o canal funciona) e, sob sincronia parcial, *precisão eventual*. Uma falsa suspeita causa no máximo uma eleição e uma troca de época, nunca dois primários confirmando escritas: a segurança da aplicação é delegada às épocas.

C. Replicação primário-backup síncrona

Hipóteses. As mesmas do sistema; o líder do Valentão é o primário. O conjunto de N nós é fixo e o quórum $q = \lfloor N/2 \rfloor + 1$.

Escrita. O `PUT` no primário, sob uma trava que serializa as escritas, atribui o próximo `seq`, aplica e persiste, e envia `REPLICAR(época, seq, prev_epoca, chave, valor)` a todos da visão. Cada backup recusa época antiga, aplica em ordem de `seq` (idempotente: retransmissão é `DUPLICADO`) e *persiste antes de responder* `ACK`; lacuna ou histórico divergente (`prev_epoca` diferente da sua) leva à transferência de estado. Quem estoura o *timeout* sai da visão. A escrita só é confirmada se $1 + |\text{ACKs}| \geq q$; uma recusa por época maior faz o primário renunciar sem confirmar. O `GET` é servido pelo primário, que verifica ainda enxergar um quórum com ACK recente.

Por que o commit exige quórum. A formulação ingênua é "ACK de todos da visão, removendo quem estoura o *timeout*". Isso é insuficiente: um primário isolado por partição remove todos os backups por *timeout*, e "todos da visão" passa a ser o conjunto vazio; ele confirma sozinho escritas que o restante do grupo, que já elegeu outro primário, nunca verá. Com maioria, duas confirmações em épocas diferentes compartilham ao menos um nó, e um primário isolado numa minoria nunca atinge q. (A opção `-quorum 1` reproduz a regra literal.) O custo é disponibilidade: sem a maioria dos nós vivos, não há escritas nem leituras.

Fase de sincronização. Antes de servir, o novo primário (i) envia `SINC_CONSULTA(época)` a todos, o que *promete* a época; (ii) exige respostas de um quórum; (iii) adota o estado de maior posição $(\text{ult_epoca}, \text{seq})$, pedido por `PEDIR_ESTADO`; (iv) empurra esse estado a quem estiver diferente (`INSTALAR_ESTADO`); e só então fica `pronto`; antes, responde `lider_nao_pronto`. A posição compara primeiro a época, como no Raft [5]: uma escrita de época maior vence a cauda *não confirmada* de um primário deposto.

Nenhuma escrita confirmada se perde. Seja w confirmada na época e: w está no disco de um conjunto Q_w com $|Q_w| \geq q$ (o primário e os backups que persistiram antes do ACK). Seja $e' > e$ a época de um novo primário, cujo quórum de sincronização é Q_s. Por maioria, $Q_w \cap Q_s \neq \emptyset$. (a) Todo nó de Q_w aceitou w *antes* de prometer e': depois da promessa ele recusa `REPLICAR` de época menor, logo o primário de e não consegue mais acumular ACKs, e o estado que o nó da interseção informa já contém w. (b) O primário adota a maior posição entre as respostas; como a posição de um nó que contém w é $\geq$ a de w, o estado adotado contém w (a menos que o nó tenha instalado um estado mais avançado, que por indução sobre as épocas também contém w). (c) A época e o líder reconhecido são persistidos, então a promessa sobrevive a crash. A conclusão vale *enquanto algum nó de Q_w sobrevive e participa da sincronização*; se todos os que têm w perdem o disco ou morrem, e os sobreviventes formam quórum, w se perde (Seção VIII). Escritas *não* confirmadas têm resultado indeterminado: podem ficar em alguns backups e ser adotadas pelo próximo primário; só respostas `ok` são garantias, e o cliente repete o `PUT`, que é idempotente.

Reingresso. Um nó que volta de crash (ou foi removido por *timeout*) vê, no *heartbeat*, que não está na visão e envia `ENTRAR` com sua posição; sob a trava de escrita, o primário lhe empurra o estado, se preciso, e o inclui.

Segurança e vivacidade. *(S1)* no máximo um líder efetivo por época; *(S2)* toda escrita confirmada está no estado de todo líder posterior e, portanto, no estado final das réplicas vivas; *(S3)* um primário deposto ou isolado não confirma (*fencing* por época mais quórum); *(S4)* réplicas em dia convergem. *Vivacidade*, sob sincronia parcial e maioria viva: um líder pronto aceita escritas e as confirma; um nó recuperado entra na visão e fica em dia.

D. Complexidade de mensagens

Valentão. Garcia-Molina [2] dá $N-2$ mensagens no melhor caso (o maior id vivo detecta e anuncia aos $N-2$ demais vivos) e $O(N^2)$ no pior. Nossos contadores contam mensagens *enviadas*, inclusive as endereçadas ao nó morto. Se só o nó m detecta a falha do líder N, os iniciadores são $m, \ldots, N-1$ (cada `OK` faz o nó maior abrir a sua eleição). O iniciador i envia $N-i$ `ELEICAO` (a um dos quais, o N, ninguém responde) e recebe $N-1-i$ `OK`; somando,

$$\textstyle\sum_{i=m}^{N-1}(N-i) = \frac{(N-m)(N-m+1)}{2},$$
$$\textstyle\sum_{i=m}^{N-1}(N-1-i) = \frac{(N-m)(N-m-1)}{2},$$

e o vencedor $(N-1)$ anuncia `COORDENADOR` a todos os $N-1$ pares, vivos ou não. O total é

$$M(N,m) = (N-m)^2 + (N-1). \tag{1}$$

O melhor caso ($m = N-1$) é $M = N$: um `ELEICAO` ao morto mais $N-1$ `COORDENADOR`, equivalente aos $N-2$ de Garcia-Molina entre os vivos. O pior ($m = 1$) é $M = N(N-1) = \frac{N(N-1)}{2} + \frac{(N-1)(N-2)}{2} + (N-1)$ (`ELEICAO`+`OK`+`COORDENADOR`), $O(N^2)$. **Primário-backup.** Uma escrita custa um `REPLICAR` e um `ACK` por backup: $2(N-1)$. **Heartbeat.** $2(N-1)$ mensagens por intervalo τ_{hb} (`HEARTBEAT` mais `HB_ACK`), isto é, $O(N)$. Esses valores são comparados com os contadores na Seção VII.

V. Testes e verificação

Há 50 testes automatizados (`pytest`): 40 do sistema (cerca de 40 s) e 10 do módulo de estatística (IC, percentis) usado nos experimentos. Dos 40, 25 são unitários, sem rede, para a réplica (ordem, idempotência, lacuna, histórico divergente, *snapshot*), a persistência (recuperação, WAL truncado), a injeção de falhas

(semente, partição, filtro), as mensagens, os contadores e o verificador (inclusive a detecção de violações forjadas); e 15 de integração, com processos reais em localhost. Estes cobrem: Valentão normal ($N = 5$, líder 5, um líder por época), morte do líder (eleito o 4), retorno do 5 e várias falhas (5, 4, 3); heartbeat (suspeita do líder morto; falsa suspeita por partição $1 \leftrightarrow 5$ sem crash, registrada e revogada); replicação normal (réplicas idênticas, custo $2(N-1)$, redirecionamento); backup morto sai da visão; **primário morto durante carga** sem perda de escritas confirmadas; nó recuperado re-sincroniza; reinício do cluster inteiro a partir do disco; ausência de quórum não confirma; **fencing** (primário isolado não confirma e, ao curar, renuncia e descarta sua escrita); e descarte aleatório de mensagens.

Cada nó escreve um log JSONL (`t`, `p=perf_counter` para intervalos, nó, incarnação, evento). O verificador (`main.py verificar`) ordena os logs e checa: **I1** no máximo um `lider_pronto` por época; **I2** a época de cada nó nunca diminui; **I3** toda `escrita_confirmada` veio de um nó que já era líder pronto naquela época; **I4** dois `seq` confirmados iguais têm o mesmo conteúdo; **I5** toda escrita confirmada está no estado final de cada réplica viva (o valor final é o da última confirmada ou de uma escrita posterior); **I6** as réplicas vivas convergiram. Todos os testes de integração terminam com o verificador (I1–I6). Mais de um `coordenador_anunciado` por época é apenas aviso, possível em partição.

VI. Cenários experimentais e métricas

Protocolo. A campanha é automatizada (`python main.py experimentos`, código em `src/experimentos.py`). Cada réplica é um cluster novo de processos locais; cada configuração tem $n = 10$ réplicas independentes, com sementes registradas em `metadados.json` (`exp·10000+config·100+réplica`, que também semeiam a injeção de falhas) e dados brutos em `resultados/brutos/`. Reportamos a média com desvio-padrão s e IC bilateral de 95% pela t de Student com $n - 1 = 9$ graus de liberdade (a variância populacional é desconhecida e n é pequeno):

$$\bar{x} \pm t_{0{,}975;9} \frac{s}{\sqrt{n}}, \qquad t_{0{,}975;9} = 2{,}262. \tag{2}$$

Nas latências, p50/p95/p99 são calculados por réplica e a tabela traz a média (e o IC) entre réplicas. Toda réplica termina com o verificador (I1–I6) e, nas de falha, com a checagem do lado do cliente de que toda escrita confirmada está em todas as réplicas vivas. Intervalos usam `perf_counter` (campo `p` dos logs). Como timers do Windows têm tique de 15,6 ms, que distorcia o atraso injetado e os *timeouts* curtos, o processo do nó chama `timeBeginPeriod(1)`. A fase do crash em relação ao *heartbeat* é sorteada por réplica. Metadados: Tabela I.

E1: Valentão vs. N. Mata-se o líder (`Popen.kill`) com $N \in \{3, 5, 7, 9\}$. Mensagens: variação dos contadores de `ELEICAO`, `OK` e `COORDENADOR` enviadas. Tempos, decompostos em detecção (*kill*→1ª suspeita), eleição (1ª `eleicao_iniciada`→ `COORDENADOR`) e sincronização (→`lider_pronto`). Quatro variantes: *melhor* (só o nó $N-1$ detecta), *médio* (só $\lceil N/2 \rceil$), *pior* (só o nó 1; os demais têm $\tau_{fd} = 30$ s) e *natural* (todos detectam por *timeout*); comparam-se com (1).

Tabela I
Metadados dos ensaios.

Hardware	Intel Core i5-13420H, 12 núcleos lógicos, 15,7 GB RAM
SO / software	Windows 11 (10.0.26200), Python 3.11.9, numpy 2.3.3, matplotlib 3.10.6, pytest 8.4.2
Execução	processos locais em 127.0.0.1, sem contêineres; 37,8 min no total; 10 réplicas/config.
Padrão	$\tau_{hb} = 0{,}1$ s, $\tau_{fd} = 0{,}5$ s, $\tau_{rpc} = 0{,}3$ s (salvo se variado)
E1	1 s de *heartbeats* e 3 escritas antes do crash; ≈1 s de eleição + 0,5 s
E2	1,5 s de aquecimento descartado; 6 s medidos
E3a	1,5 s de escrita antes do crash; até a eleição + 1 s; $\tau_{hb} = \tau_{fd}/5$
E3b	2 s de aquecimento com a perda ativa; 15 s de janela
E4	5 escritas antes; 4 s de partição; depois, convergência

E2: escrita. Um cliente síncrono em laço fechado faz `PUT` no primário, com $N \in \{3, 5, 7, 9\}$ e, com $N = 5$, atraso fixo injetado em todas as mensagens entre nós (inclusive o `ACK`) de 0, 5 e 20 ms. Latência medida no cliente (inclui abrir a conexão); vazão; mensagens por escrita (`REPLICAR`+`ACK` enviados por escrita no servidor), a comparar com $2(N-1)$.

E3: *failover* e falsas suspeitas. (a) $N = 5$, escritor contínuo, crash do primário, $\tau_{fd} \in \{0{,}25; 0{,}5; 1; 2\}$ s. A *janela* de indisponibilidade é o maior intervalo entre duas escritas confirmadas consecutivas que cobre o crash, vista pelo cliente (timeout de 0,5 s por tentativa, com retentativa). (b) $N = 5$, *sem* crash, perda aleatória de 0, 5, 10 e 20% das mensagens entre nós e um escritor leve (um `PUT` a cada 50 ms); como ninguém cai, toda suspeita é falsa. Contam-se suspeitas do detector, eleições espúrias (motivo ≠ início) e escritas confirmadas em 15 s. O jitter não foi variado.

E4: partição. $N = 5$; isola-se o primário (nó 5) dos demais por ≈4 s enquanto um cliente tenta escrever nele a cada 50 ms e outro escreve no lado majoritário; depois a partição é curada. Medem-se as confirmações de cada lado e os tempos de reconciliação.

VII. Resultados

A. E1: tempo e mensagens da eleição

A Tabela II mostra que a contagem de mensagens é determinística: o desvio-padrão foi zero nas 160 réplicas, e os valores coincidem exatamente com (1). A variante *natural* coincide com o *pior caso*: com detecção por *timeout* o nó 1 também detecta e a cascata de `OK` faz todos os $N-1$ vivos abrirem eleição. Portanto, o comportamento típico do Valentão aqui já é quadrático (Fig. 1a).

Nenhuma fase do tempo cresce com N. A detecção (≈ 0,47–0,51 s) é τ_{fd} menos a fase do último *heartbeat*. Eleição e sincronização duram ≈ $\tau_{rpc} = 0{,}3$ s, pois as mensagens de

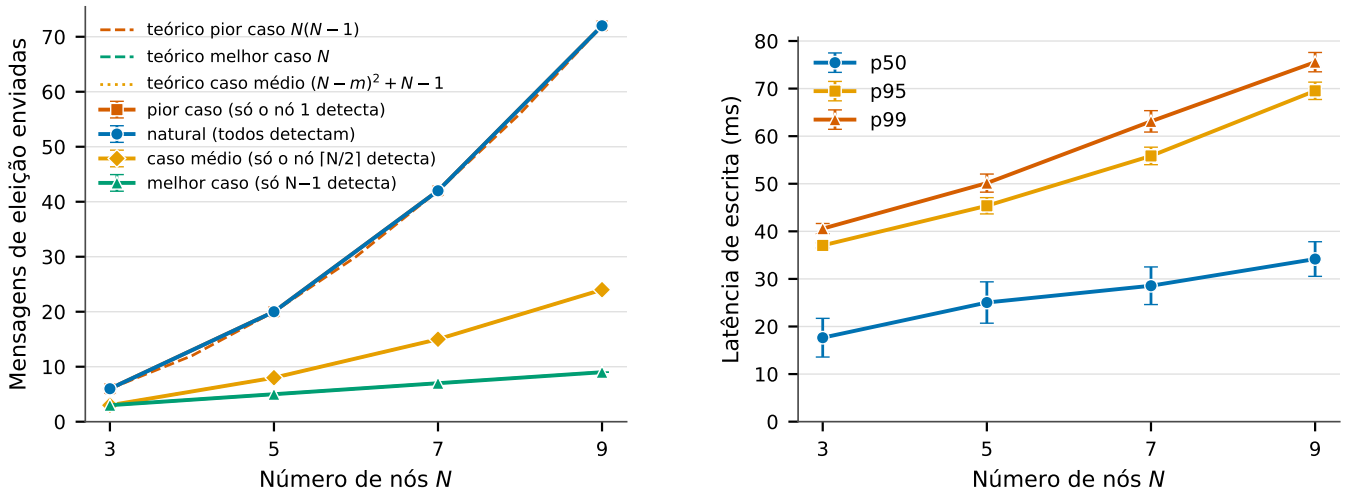


Figura 1. (a) E1: mensagens da eleição vs. N para as quatro variantes e as curvas teóricas. (b) E2: latência de escrita (p50/p95/p99) vs. N.

Tabela II
E1: MENSAGENS ENVIADAS NA ELEIÇÃO (DESVIO ZERO; IGUAIS À TEORIA). TEORIA: N, $(N-m)^2+N-1$ COM $m=\lceil N/2\rceil$ E $N(N-1)$.

N	Melhor	Médio	Pior	Natural
3	3	3	6	6
5	5	8	20	20
7	7	15	42	42
9	9	24	72	72

Tabela III
E1 (*natural*): TEMPOS EM S, MÉDIA ± IC95%.

N	Detecção	Eleição	Sincr.	*Kill*→pronto
3	$0{,}510 \pm 0{,}024$	$0{,}303 \pm 0{,}004$	$0{,}301 \pm 0{,}006$	$1{,}113 \pm 0{,}025$
5	$0{,}482 \pm 0{,}025$	$0{,}317 \pm 0{,}013$	$0{,}304 \pm 0{,}008$	$1{,}104 \pm 0{,}033$
7	$0{,}470 \pm 0{,}030$	$0{,}314 \pm 0{,}005$	$0{,}301 \pm 0{,}007$	$1{,}085 \pm 0{,}034$
9	$0{,}476 \pm 0{,}020$	$0{,}311 \pm 0{,}008$	$0{,}311 \pm 0{,}010$	$1{,}098 \pm 0{,}024$

uma rodada vão em paralelo e a rodada só termina quando a chamada ao nó morto estoura o *timeout*; no Windows, conectar a uma porta fechada leva ∼2 s (medido), então só o *timeout* encerra a chamada. O custo $O(N^2)$ aparece em mensagens, não em latência, em localhost.

B. E2: latência e vazão da escrita

Tabela IV
E2: LATÊNCIA (MS), VAZÃO (OPS/S) E MENSAGENS POR ESCRITA; MÉDIA ± IC95% DAS 10 RÉPLICAS.

Config.	Vazão	p50	p95	p99	Msg./esc.
N=3, 0 ms	$50{,}2 \pm 3{,}1$	$17{,}7 \pm 4{,}1$	$37{,}0 \pm 0{,}5$	$40{,}6 \pm 1{,}1$	4
N=5, 0 ms	$37{,}8 \pm 3{,}1$	$25{,}0 \pm 4{,}3$	$45{,}4 \pm 1{,}7$	$50{,}1 \pm 1{,}9$	8
N=7, 0 ms	$32{,}3 \pm 2{,}3$	$28{,}6 \pm 4{,}0$	$55{,}9 \pm 1{,}8$	$63{,}1 \pm 2{,}2$	12
N=9, 0 ms	$26{,}2 \pm 1{,}6$	$34{,}2 \pm 3{,}6$	$69{,}5 \pm 1{,}8$	$75{,}6 \pm 2{,}0$	16
N=5, 5 ms	$27{,}7 \pm 0{,}9$	$35{,}0 \pm 2{,}1$	$55{,}7 \pm 1{,}9$	$61{,}8 \pm 1{,}3$	8
N=5, 20 ms	$14{,}1 \pm 0{,}3$	$71{,}8 \pm 2{,}1$	$88{,}8 \pm 1{,}8$	$93{,}2 \pm 2{,}6$	8

Mensagens por escrita foram *exatamente* $2(N-1)$, com desvio zero nas 60 réplicas. A latência cresce com N (p50 de 17,7 para 34,2 ms de $N=3$ a 9), porque o primário abre $N-1$ conexões e trata $N-1$ `ACK` no mesmo laço de eventos, e com o atraso injetado: a cada escrita ele soma $\approx 2\times$ o atraso nominal (ida do `REPLICAR` e volta do `ACK`, com os backups em paralelo). A vazão cai na mesma proporção, pois há um cliente síncrono e o primário serializa as escritas. A latência absoluta é alta ($\geq$ 17 ms) porque cada mensagem abre uma conexão TCP, o que no Windows é caro; vale a tendência, e os valores absolutos não se transferem a Linux ou a contêineres.

C. E3: failover e falsas suspeitas

Tabela V
E3A: INDISPONIBILIDADE APÓS O CRASH DO PRIMÁRIO ($N=5$), EM S.

τ_{fd}	Detecção	*Kill*→pronto	Janela (média ± IC)	p50	p95
0,25	0,255	0,859	$1{,}29 \pm 0{,}04$	1,30	1,34
0,5	0,522	1,137	$1{,}36 \pm 0{,}21$	1,18	1,79
1,0	1,052	1,664	$1{,}96 \pm 0{,}18$	1,78	2,26
2,0	2,189	2,803	$3{,}10 \pm 0{,}22$	2,92	3,54

A janela vista pelo cliente é $\approx \tau_{fd}$ mais eleição e sincronização ($\approx 2\tau_{rpc}$) mais a retentativa do cliente: a detecção segue τ_{fd}, e há um piso fixo de ∼0,6 s de eleição e sincronização (esperam o *timeout* do nó morto; τ_{rpc} não foi variado). A janela excede o *kill*→pronto em 0,2 a 0,45 s, pois a tentativa ao primário morto só falha após o *timeout* de 0,5 s do cliente. Esse timeout é o piso que torna as janelas em $\tau_{fd}=0{,}25$ e 0,5 s quase iguais, e a retentativa gera a bimodalidade (p50 abaixo da média em 0,5 e 1 s). Nenhuma escrita falhou (o cliente repete) e nenhuma confirmada se perdeu.

Sem perda, não houve falsa suspeita. A taxa cresce *mais que linearmente* com a perda (Tabela VI, Fig. 2b): um seguidor só suspeita após ∼5 *heartbeats* seguidos sem chegar, e a perda também derruba os `ACK` e as mensagens de eleição, o que

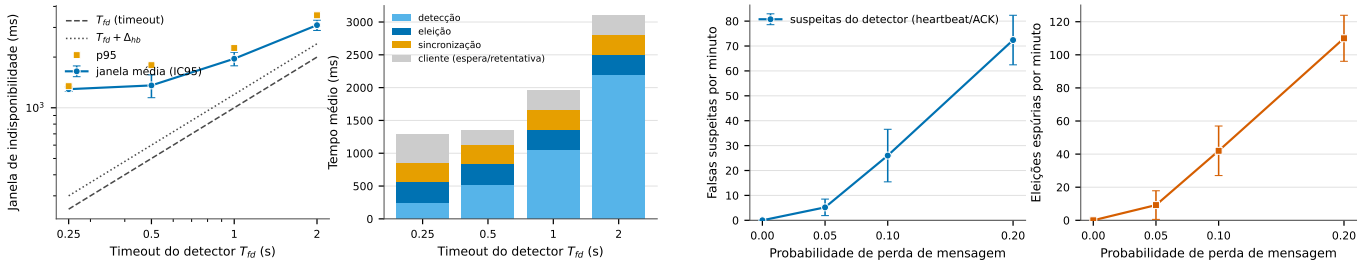


Figura 2. (a) E3a: janela de indisponibilidade e decomposição vs. τ_{fd}. (b) E3b: suspeitas falsas vs. perda de mensagens.

Tabela VI
E3B: SEM CRASH, $N = 5$, JANELA DE 15 S. SUSPEITAS DO DETECTOR E ELEIÇÕES ESPÚRIAS POR MINUTO (MÉDIA $\pm$ IC95%).

Perda	Susp./min	Eleições esp./min	Troca de líder	Escritas ok
0	0	0	0	188
0,05	$5{,}2 \pm 3{,}3$	$9{,}2 \pm 8{,}6$	0	75
0,10	$26{,}0 \pm 10{,}6$	$42{,}0 \pm 15{,}0$	0,4	47
0,20	$72{,}4 \pm 10{,}0$	110 ± 14	0,8	15

realimenta o processo. Quase não há troca de líder: o líder, ao receber `ELEICAO`, responde com *heartbeat* em vez de reeleger, e as épocas impedem dois primários. O efeito visível é na vazão: cada `REPLICAR` ou `ACK` perdido custa um τ_{rpc} (0,3 s) e a remoção do backup da visão (reentrada por `ENTRAR`), e o commit exige quórum; assim, as escritas confirmadas caem de 188 para 15 em 15 s com 20% de perda, sem perder nenhuma confirmada.

D. E4: partição do primário

Tabela VII
E4: PRIMÁRIO (NÓ 5) ISOLADO POR $\approx$4 S ($N = 5$, 10 RÉPLICAS).

Métrica	Média $\pm$ IC95%
Tentativas de escrita no primário isolado	$55{,}7 \pm 1{,}4$
Confirmadas pelo primário isolado (`sem_quorum`)	0 ± 0
Maioria: nó 4 pronto após o isolamento (s)	$1{,}131 \pm 0{,}014$
Confirmadas no lado majoritário (de 20)	20 ± 0
Nó 5 renuncia após curar (s)	$0{,}261 \pm 0{,}064$
Nó 5 retoma a liderança (s)	$0{,}487 \pm 0{,}090$
Convergência total após curar (s)	$0{,}627 \pm 0{,}098$
Escritas órfãs no estado final	0

O primário isolado não confirmou nenhuma das $\sim$56 tentativas, todas respondidas com `sem_quorum`; a maioria elegeu o nó 4, que ficou pronto em $\sim$1,13 s e confirmou todas as 20 escritas. Ao curar, o nó 5 renunciou pela época maior, retomou a liderança (maior id) e tudo convergiu, sem escritas órfãs. É o *split-brain* da Seção IV-A resolvido pelo quórum, na prática.

Correção em todos os experimentos. Nas 310 réplicas de falha, o verificador reportou 0 violações (I1–I6) e a checagem do cliente, 0 escritas confirmadas perdidas.

VIII. LIMITAÇÕES

Domínios de falha. Processos ou contêineres numa só máquina não são independentes (Seção II): não há falha correlacionada, latência real de rede nem perda de disco, e os tempos refletem um único escalonador; os valores absolutos não se transferem a outro hardware. **Docker.** O `compose.yaml` não foi executado por nós (Seção II). **Disponibilidade.** O quórum de maioria sacrifica disponibilidade: sem a maioria dos nós vivos, não há escritas nem leituras. **Perda possível.** Uma escrita confirmada se perde se todos os nós que a têm morrem ou perdem o disco e os sobreviventes formam quórum; `kill` preserva o disco, e `-fsync` cobre queda da máquina. **Leituras.** O `GET` é no primário, com *lease* de τ_{fd}; entre a perda de liderança e esse limite, uma leitura obsoleta é possível, e depende da sincronia parcial. **Desempenho.** Uma conexão TCP por mensagem e escritas serializadas no primário privilegiam a simplicidade, não a vazão; o conjunto de nós é fixo (sem reconfiguração); não se toleram falhas bizantinas.

IX. CONCLUSÃO

Integramos Valentão, primário-backup síncrono e detector por *heartbeat* em um armazenamento replicado cuja segurança repousa em três peças: quórum de maioria no commit, épocas e líder reconhecido persistidos e fase de sincronização pós-eleição. O Valentão pode produzir dois líderes sob partição, e o heartbeat pode suspeitar de nós vivos; ambos custam apenas vivacidade, pois só o lado majoritário conclui a sincronização e os demais são recusados por época. As propriedades são checadas por testes de integração e pelo verificador de logs, e a campanha as confirmou: em 310 réplicas de falha, houve 0 violações e 0 escritas confirmadas perdidas. O Valentão custou N mensagens no melhor caso e $N(N-1)$ no pior (também o caso natural), sem que o tempo de eleição crescesse com N; a escrita custou $2(N-1)$ mensagens; a janela de indisponibilidade foi de 1,29 a 3,10 s para τ_{fd} de 0,25 a 2 s; com 20% de perda, as escritas confirmadas caíram de 188 para 15 em 15 s, sem perda de dados; e o primário isolado não confirmou nada.

DIVISÃO DE TRABALHO

Caique Pinheiro Andrade: implementação dos algoritmos e da infraestrutura de comunicação, e execução da campanha experimental. *Luiz Henrique Marques Gonçalves:* modelo do sistema e argumentos de correção, análise dos resultados e redação do relatório.

REFERÊNCIAS

[1] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed. distributed-systems.net, 2023.

[2] H. Garcia-Molina, "Elections in a distributed computing system," *IEEE Trans. Computers*, vol. C-31, no. 1, pp. 48–59, 1982.

[3] N. Budhiraja, K. Marzullo, F. B. Schneider, and S. Toueg, "The primary-backup approach," in *Distributed Systems*, 2nd ed., S. Mullender, Ed. ACM Press, 1993, pp. 199–216.

[4] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," *J. ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[5] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2014, pp. 305–319.